

Here is your ultimate, copy-paste deployment cheat sheet. Save this somewhere handy. Whenever you want to swap out the website running at <https://deenihalaka.redwanshub.studio>, just run through these commands in order.

1. Stop the Current Server

Always shut down the active container before deleting files to release the port and prevent memory crashes.

```
Bash
cd /var/www/website
sudo docker compose down
```

2. Purge the Old Code (But Keep Docker Files)

You need to delete the old React/Next.js files, but **you must keep your Dockerfile and docker-compose.yml**.

Run this exact command to safely wipe the standard Next.js source code while keeping your infrastructure files completely untouched:

```
Bash
sudo rm -rf app components hooks lib public styles node_modules .next package.json
pnpm-lock.yaml next.config.mjs postcss.config.mjs tailwind.config.ts tsconfig.json
README.md components.json
```

3. Upload Your New Website

At this step, transfer your new Next.js project files into `/var/www/website` using SFTP, Git, or however you prefer to move your files to the Raspberry Pi.

4. Inject the Standalone Rule

Your new website must be told to compile efficiently to protect your 4GB of RAM. Copy and paste this entire block at once to instantly generate the perfect configuration file:

```
Bash
cat << 'EOF' > /var/www/website/next.config.mjs
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'standalone',
  typescript: {
    ignoreBuildErrors: true,
  },
  eslint: {
    ignoreDuringBuilds: true,
  },
};
export default nextConfig;
EOF
```

5. Force-Rebuild and Launch

You must use the `--build` flag so Docker knows to completely ignore its old cache and compile your brand-new files from scratch.

```
Bash
sudo docker compose up -d --build
```

Troubleshooting Quick-Reference

How to check if the new site actually booted:

(This prints the live terminal output from inside the container)

```
Bash  
sudo docker logs -f deenihalaka_website
```

(Press Ctrl+C to exit the log viewer)

If the build fails on step pnpm install:

It means your new website uses npm instead of pnpm. Open your Dockerfile and change RUN npm install -g pnpm and RUN pnpm install to just standard RUN npm install.